MLOps Tool Setup & Debugging — Internal POE

DevOps Engineer UAT Dry Run & Tool Integration Summary

Candidate Name:	Abhi Trambadiya	**Course / Sem:**	Lab MLDevOps (Sem 07)
Roll Number:	A36 (Div A, Roll 36)	**PRN:**	2324000344
Role Assumed:	DevOps Engineer (UAT Dry Run)	**GitHub Account:**	abhitrambadiya
Execution Date:	October 30, 2026	**Evaluation:**	Internal Practical Oral Exam (POE)

1. Executive Summary & Purpose

Modern Machine Learning systems deployed to production face catastrophic failures when infrastructure, dependencies, and input data shift unexpectedly. As part of the MLOps Internal POE, this project establishes a hardened UAT (User Acceptance Testing) dry run workflow across the entire DevOps/MLOps stack: VSCode and Git version control, lightweight containerization with Docker, automated unit testing with Pytest, declarative CI/CD orchestration with Jenkins, and data drift observability with Evidently AI.

2. Tool Implementation & Installation Summary Report

Tool Name	Commands Used	Installation / Run Status

VSCode + Git	`git init`
	`git add .`
	`git commit -m 'feat: ...'`
	`git push origin main`
		Successful
		Structured workspace & GitHub integration active.

Pytest	`pip install pytest` `pytest tests/ -v`	**Successful** 2/2 test cases passed with zero regressions.

Docker Containerization	`docker pull python:3.11-alpine`
	`docker build -t mlops-hello-poe .`
	`docker run --rm mlops-hello-poe`
		Successful
		Lightweight 58MB image built and validated.

Jenkins CI/CD	`Jenkinsfile (Declarative)` `Stages: Checkout → Verify → Test → Build → Run → Monitor`	**Successful** Pipeline validated for automated deployment.
Evidently AI Monitoring	`pip install evidently` `python3 monitoring/monitor.py`	**Successful** HTML report & JSON drift metrics exported.

3. Execution Output & Verification Logs

Verification Stage	Observed Terminal Output

Step 1: Application Execution (hello.py)	`$ python3 hello.py`
	`Hello, DevOps Engineer! Welcome to MLOps Lab Internal POE.`
	`Sample Feature Calculation Score: 11.15`

Step 2: Pytest Automation Execution	`$ pytest tests/ -v`
	`collected 2 items`
	`tests/test_hello.py::test_get_greeting PASSED [ 50%]`
	`tests/test_hello.py::test_calculate_feature_score PASSED [100%]`
	`======= 2 passed in 0.01s =======`

Step 3: Docker Alpine Build & Run	`$ docker build -t mlops-hello-poe:latest .`
	`[+] Building 29.7s (10/10) FINISHED`
	`naming to docker.io/library/mlops-hello-poe:latest`
	`$ docker run --rm mlops-hello-poe:latest`
	`Hello, DevOps Engineer! Welcome to MLOps Lab Internal POE.`
	`Sample Feature Calculation Score: 11.15`

Step 4: Pytest Inside Docker Container	`$ docker run --rm mlops-hello-poe:latest pytest tests/ -v`
	`platform linux -- Python 3.11.18, pytest-9.2.0`
	`tests/test_hello.py::test_get_greeting PASSED [ 50%]`
	`tests/test_hello.py::test_calculate_feature_score PASSED [100%]`
	`======= 2 passed in 0.01s =======`

Step 5: Evidently AI Drift Monitoring	`$ python3 monitoring/monitor.py`
	`[INFO] Computing Data Drift and Quality metrics across datasets...`
	`[SUCCESS] Evidently Drift HTML Report:`
	`monitoring/reports/evidently_drift_report.html`
	`[SUCCESS] Evidently Drift JSON Metrics:`
	`monitoring/reports/evidently_drift_report.json`

4. Jenkins Declarative Pipeline Architecture

The automated Jenkins pipeline triggers on push/PR events to execute continuous integration stages: **Stage 1: Checkout Source** from Git → **Stage 2: Tool Verification** (Python, Docker, Git) → **Stage 3: Pytest Automation** → **Stage 4: Docker Build & Package** → **Stage 5: Container UAT Dry Run** → **Stage 6: Evidently Model & Data Drift Validation**.

5. Outcomes & Production Readiness

1. Zero Environmental Discrepancy: Packaging with Alpine Python ensures deterministic container runtimes across local macOS, Windows lab machines, and Linux CI/CD runners.
2. Shift-Left Quality Gates: Automated Pytest test execution catches logical flaws prior to containerization and packaging.
3. Statistical Data Observability: Implementing Kolmogorov-Smirnov (KS) tests via Evidently AI guarantees that production data divergence is detected immediately prior to model degradation.